

Busca em profundidade

Algoritmos e Estruturas de Dados 2

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

15 de setembro de 2026

Do modelo ao algoritmo

Já temos o modelo

- grafo = vértices + arcos;
- matriz de adjacências: espaço V^2 , adjacência em tempo constante;
- listas de adjacência: espaço $V + A$, percurso de vizinhos proporcional ao grau.

Sabemos *guardar* um grafo.

Falta o algoritmo

Nenhuma das duas representações responde, sozinha, a perguntas como:

- existe caminho de v até w ?
- quais vértices são alcançáveis a partir de s ?
- o grafo tem ciclo? em quantos pedaços ele se divide?

Todas exigem *percorrer* o grafo.

Para aprofundar: representações de grafos em [5], cap. *Estruturas de dados para grafos*, e [4], §22.1.

Definições

Um **caminho** de v a w é uma sequência de vértices $v = u_0, u_1, \dots, u_k = w$ em que $u_i - u_{i+1}$ é arco do grafo para todo i . Dizemos que w é **alcançável** a partir de v se existe tal caminho.

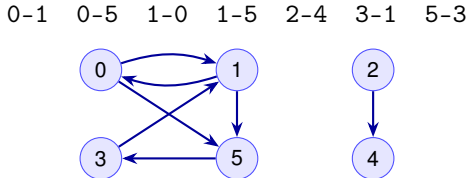
Por que isso é difícil por inspeção

O número de sequências de vértices cresce exponencialmente com V . Precisamos de um percurso que visite cada vértice *no máximo uma vez* — e que, mesmo assim, não deixe nenhum vértice alcançável de fora.

Para aprofundar: caminhos, passeios e conexidade em [6] (em português) e [4], apêndice B.4.

O grafo desta aula

O mesmo da aula anterior: vértices 0..5 e o conjunto de arcos



Convenção da aula: os vizinhos de um vértice são sempre examinados em ordem crescente. Isso torna o percurso *determinístico* e as simulações reproduzíveis.

Para aprofundar: grafo-exemplo e convenções de código de [5].

A ideia da busca em profundidade

Busca em profundidade, em uma frase

Ao chegar a um vértice ainda não visitado, **marque-o** e siga imediatamente por um arco que leve a um vértice ainda não marcado; quando não houver nenhum, **volte** ao vértice de onde você veio e continue de lá.

- “Profundidade” é a prioridade: avançar sempre que possível, recuar só quando não há saída nova.
- A marcação é o que impede voltar a andar em círculos — o fio que Teseu desenrola no labirinto.
- A recursão da linguagem C já implementa o “volte ao vértice de onde você veio”: é a pilha de chamadas.

Para aprofundar: a ideia da busca em profundidade em [5], cap. *Busca em profundidade*; [18], §4.1; [14], §3.5.

Terminação

Um vértice é marcado no máximo uma vez, e só examinamos os arcos que saem de um vértice no momento em que ele é marcado. Logo cada arco é examinado no máximo uma vez: o processo é finito.

Corretude (alcançabilidade)

Ao final da busca iniciada em s , um vértice está marcado *se e somente se* é alcançável a partir de s .

Esboço: todo vértice marcado foi atingido por uma sequência de arcos a partir de s , logo é alcançável.

Reciprocamente, se algum vértice alcançável ficasse desmarcado, haveria no caminho de s até ele um primeiro arco $v-w$ com v marcado e w desmarcado — impossível, pois ao marcar v examinamos todos os seus vizinhos.

Para aprofundar: prova formal (teorema do caminho branco) em [4], §22.3; a DFS como ferramenta de tempo linear nasce em [19].

```
static int cnt;
static int pre[maxV];    /* pre[v] = ordem de visita de v; -1 = nao visitado */

/* Visita todos os vertices ainda nao visitados que sao alcancaveis a
partir de v. Supoe que G e representado por matriz de adjacencias. */
static void dfsR( Graph G, vertex v) {
    pre[v] = cnt++;      /* marca v */
    for (vertex w = 0; w < G->V; ++w) /* examina os vizinhos de v */
        if (G->adj[v][w] != 0)
            if (pre[w] == -1) /* w ainda nao visitado */
                dfsR( G, w); /* avanca */
}                          /* fim: volta ao chamador */
```

O vetor `pre[]` faz dois papéis ao mesmo tempo: diz se o vértice já foi visitado e *quando* isso aconteceu.

Para aprofundar: código-fonte e variantes em [5]; implementação em C em [17], cap. 18.

```
static void dfsR( Graph G, vertex v) {  
    pre[v] = cnt++;  
    for (link a = G->adj[v]; a != NULL; a = a->next) {  
        vertex w = a->w;  
        if (pre[w] == -1)  
            dfsR( G, w);  
    }  
}
```

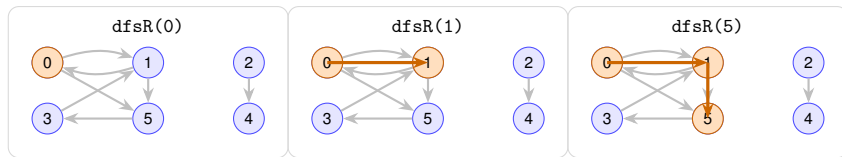
A diferença que importa

O corpo do algoritmo é o mesmo; muda apenas *como* se enumeram os vizinhos: na matriz, sempre V testes; nas listas, tantos passos quanto o grau de saída. Como GRAPHinsertArc() insere no *início* da lista, os vizinhos saem na ordem inversa à de inserção e a busca a partir de 0 visita 0, 5, 3, 1.

Para aprofundar: [5]; [18], §4.1.

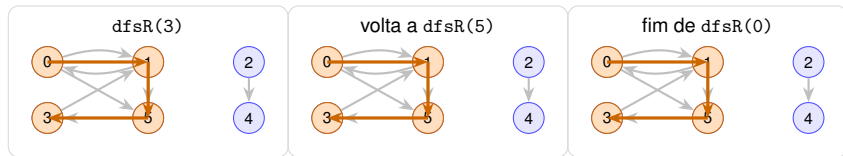
Simulando a busca

Passo a passo a partir de 0 — primeira metade



Em `dfsR(1)` o primeiro vizinho é 0, já marcado: nada acontece. O segundo é 5, que ainda não foi visitado — a busca avança. Laranja = marcado; laranja grosso = arco por onde a busca avançou.

Para aprofundar: simulações passo a passo equivalentes em [5] e [17], cap. 18.



O único vizinho de 3 é 1, já marcado: a busca recua até 0, cujo segundo vizinho (5) também já está marcado.

Resultado: os vértices alcançáveis a partir de 0 são $\{0, 1, 5, 3\}$; **2 e 4 não são alcançáveis.**

Para aprofundar: alcançabilidade e busca a partir de uma raiz em [5]; [4], §22.3.

vértice v	0	1	2	3	4	5
<code>pre[v]</code>	0	1	-1	3	-1	2

- A ordem 0, 1, 5, 3 em que os vértices receberam número é a **ordem de pré-ordem** (ou *ordem de descoberta*).
- Ela é atribuída na *entrada* da chamada recursiva, antes de examinar os vizinhos.
- Os vértices que permanecem com -1 são exatamente os *não alcançáveis* a partir da raiz da busca.

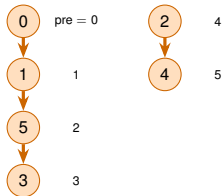
Para aprofundar: pré-ordem e carimbos de tempo (*timestamps* $d[v]$ e $f[v]$) em [4], §22.3; numeração `pre[]` em [5].

Varredura completa e floresta DFS

```
/* Visita TODOS os vertices do grafo, em uma ou mais buscas. */  
void GRAPHdfs( Graph G) {  
    cnt = 0;  
    for (vertex v = 0; v < G->V; ++v)  
        pre[v] = -1;  
    for (vertex v = 0; v < G->V; ++v)  
        if (pre[v] == -1)          /* v ainda nao alcancado por busca anterior */  
            dfsR( G, v);          /* começa uma nova busca com raiz v */  
}
```

Cada iteração do segundo laço que dispara `dfsR()` inicia uma **nova busca**, com uma nova **raiz**. No nosso grafo há duas: o vértice 0 e o vértice 2.

Para aprofundar: a varredura completa (DFS) em [4], §22.3, e em [5].



Os arcos por onde a busca *avançou* formam uma floresta:

- cada árvore tem por raiz um vértice em que o laço de `GRAPHdfs()` disparou uma busca;
- os demais arcos do grafo (como 1-0 e 3-1) *não* entram na floresta — eles levam a vértices já marcados;
- a floresta tem V vértices e $V - (\text{número de raízes})$ arcos.

Na próxima aula classificaremos justamente os arcos que ficaram de fora.

Para aprofundar: floresta DFS e classificação de arcos em [4], §22.3; [17], cap. 18; origem em [19].

A versão iterativa

```
void GRAPHdfsPilha( Graph G, vertex s) {
    for (vertex v = 0; v < G->V; ++v) pre[v] = -1;    cnt = 0;
    STACKinit( G->V * G->V);                          /* pilha de vertices */
    STACKpush( s);
    while (!STACKempty()) {
        vertex v = STACKpop();
        if (pre[v] != -1) continue;                    /* ja visitado por outro caminho */
        pre[v] = cnt++;
        for (vertex w = 0; w < G->V; ++w)
            if (G->adj[v][w] != 0 && pre[w] == -1)
                STACKpush( w);
    }
}
```

A pilha substitui a pilha de chamadas. O teste `pre[v] != -1` após o pop é necessário: um vértice pode ser empilhado várias vezes.

Para aprofundar: pilha explícita: [17], cap. 18; [21], §7.3.

Recursiva, a partir de 0

Ordem de visita: 0, 1, 5, 3

Os vizinhos são examinados em ordem crescente e cada avanço acontece *antes* de o vizinho seguinte ser considerado.

Iterativa, a partir de 0

Ordem de visita: 0, 5, 3, 1

Empilhamos 1 e depois 5; o pop devolve o *último*, isto é, 5. O 1 fica esperando no fundo da pilha.

O que *não* muda

O conjunto de vértices visitados é o mesmo: $\{0, 1, 5, 3\}$. A busca em profundidade não define uma ordem única — ela depende da representação e da implementação. O que ela garante é *quais* vértices são alcançados.

Para aprofundar: comparação entre as ordens recursiva e com pilha em [17], cap. 18, e [18], §4.1.

Custo

	Matriz de adjacências	Listas de adjacência
Marcar todos os vértices	$\Theta(V)$	$\Theta(V)$
Examinar os vizinhos de v	$\Theta(V)$	$\Theta(\text{grau de saída de } v)$
Total da varredura	$\Theta(V^2)$	$\Theta(V + A)$

- Com listas o total sai da soma $\sum_v \text{grau de saída}(v) = A$: cada arco é examinado exatamente uma vez.
- Em grafos esparsos ($A \ll V^2$) a diferença é enorme — e é por isso que quase todo algoritmo de grafos deste curso pressupõe listas.
- O espaço extra é $\Theta(V)$ nos dois casos: os vetores `pre[]` e `pai[]`, mais a pilha de recursão.

Para aprofundar: análise $\Theta(V + A)$ por agregação em [4], §22.3; discussão de esparsidade em [14], §3.5, e [5].

**Aplicação: recuperar um
caminho**

```
static int pai[maxV];    /* pai[w] = vertice de onde a busca chegou a w */

static void dfsR( Graph G, vertex v) {
    pre[v] = cnt++;
    for (vertex w = 0; w < G->V; ++w)
        if (G->adj[v][w] != 0 && pre[w] == -1) {
            pai[w] = v;          /* registra o arco de arvore v-w */
            dfsR( G, w);
        }
}
```

O vetor `pai[]` é a floresta DFS armazenada em um vetor: `pai[v] == v` identifica as raízes. Custa $\Theta(V)$ de espaço e nada de tempo adicional.

Para aprofundar: vetor de pais em [5]; equivalentes $\pi[v]$ em [4], §22.3, e `edgeTo[]` em [18], §4.1.

O vetor, após a busca em 0

v	1	5	3
$\text{pai}[v]$	0	1	5

Para achar o caminho de 0 a 3, siga $\text{pai}[]$ a partir de 3 até a raiz e inverta: $3 \rightarrow 5 \rightarrow 1 \rightarrow 0$, logo o caminho é 0, 1, 5, 3.

Atenção

O caminho obtido é *um* caminho, não necessariamente o mais curto: a busca em profundidade encontrou 0, 1, 5, 3, mas 0, 5, 3 também é caminho e é mais curto. Caminhos mínimos em número de arcos são tarefa da busca em **largura**.

Para aprofundar: recuperação de caminhos ($\text{pathTo}()$) em [18], §4.1; caminhos mínimos por busca em largura em [4], §22.2.

Fechamento

Exercícios

1. Escreva `GRAPHreach(G, s)` que devolve o número de vértices alcançáveis a partir de s .
2. Adapte `dfsR()` para listas e compare a ordem de visita com a da matriz.
3. Implemente a versão iterativa com pilha e verifique a ordem 0, 5, 3, 1.
4. Escreva `GRAPHpath(G, s, t)` que imprime um caminho de s a t , ou avisa que não existe.
5. Quantas raízes tem a floresta DFS de um grafo sem arcos? E de um grafo em que todo vértice alcança todos os outros?

O tutorial em Jupyter traz esqueletos em C para os itens 1, 3 e 4.

Para aprofundar: mais exercícios em [5] e [21], §7.3.

As cinco ideias centrais

- Percorrer é o que transforma a representação em resposta: caminhos, alcançabilidade, ciclos.
- A regra da DFS: marque, avance enquanto puder, recue quando não puder.
- `pre[]` marca e numera; `GRAPHdfs()` varre o grafo todo, gerando uma **floresta**.
- Custo $\Theta(V + A)$ com listas, $\Theta(V^2)$ com matriz.
- `pai[]` guarda a floresta em um vetor e devolve um caminho — não necessariamente o mais curto.

Próxima aula: os arcos que ficaram fora da floresta — pós-ordem, classificação de arcos, detecção de ciclos e componentes.

Conceito	Onde ler
Representações e o grafo-exemplo	[5]; [4], §22.1
Caminhos, alcançabilidade	[6]; [4], ap. B.4
A ideia da DFS	[5]; [14], §3.5
Código em C (matriz e listas)	[5]; [17], cap. 18
Corretude e pré-ordem	[4], §22.3
Varredura e floresta DFS	[4], §22.3; [19]
Versão iterativa com pilha	[17], cap. 18; [21], §7.3
Custo $\Theta(V + A)$	[4], §22.3; [14], §3.5
Caminho a partir de <code>pai[]</code>	[18], §4.1

Leitura mínima da aula: [5], capítulo *Busca em profundidade*. Os demais textos servem para aprofundar cada ponto.



Jon Bentley.
Programming Pearls.
Addison-Wesley, 2 edition, 2000.



Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461, 1973.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.



J. Adrian Bondy and U. S. R. Murty.
Graph Theory, volume 244 of *Graduate Texts in Mathematics*.
Springer, 2008.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Paulo Feofiloff.

Algoritmos para grafos (em linguagem C).

Material didático online, DCC/IME-USP, 2020.

Capítulos *Grafos e Estruturas de dados para grafos*. Acesso em 01 set. 2026.



Paulo Feofiloff, Yoshiharu Kohayakawa, and Yoshiko Wakabayashi.

Uma introdução sucinta à teoria dos grafos.

Texto didático online, IME-USP, 2011.

Referência em português para as definições de teoria dos grafos. Acesso em 01 set. 2026.



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.

W. H. Freeman, 1979.



David Gries.

The Science of Programming.

Springer-Verlag, 1981.



Michael Held and Richard M. Karp.

A dynamic programming approach to sequencing problems.

Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

Algorithm 65: FIND.

Communications of the ACM, 4(7):321–322, 1961.

O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.

Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.

Software: Practice and Experience, 27(8):983–993, 1997.

O introselect: híbrido entre quickselect e mediana das medianas.



Robert Sedgewick.

Algorithms in C, Part 5: Graph Algorithms.

Addison-Wesley, 3 edition, 2002.

Fonte dos exemplos e exercícios do Capítulo 17 citados por Feofiloff.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Robert Tarjan.

Depth-first search and linear graph algorithms.

SIAM Journal on Computing, 1(2):146–160, 1972.

Artigo que estabeleceu a DFS como ferramenta algorítmica de tempo linear.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.



Nivio Ziviani.

Projeto de Algoritmos com Implementações em Pascal e C.

Cengage Learning, 3 edition, 2011.

Capítulo 7: algoritmos em grafos.